

Overfitting, Regularization & Validation

KAUST–Mawhiba IOAI Training

Week 1 · Day 4



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

April 25, 2026

1. Recap & The Generalization Problem
2. Bias-Variance Tradeoff & Detecting Overfitting
3. Regularization: Ridge (L2) & Lasso (L1)
4. Cross-Validation: K-Fold, Stratified K-Fold, Group K-Fold
5. Hyperparameter Tuning: Grid & Random Search
6. The Full ML Pipeline

Day 2: Prepared data (visualize, clean, scale, split, feature engineer).

Day 3: Built models (Linear/Logistic Regression) and evaluated them (metrics, baselines).

But there's a problem...

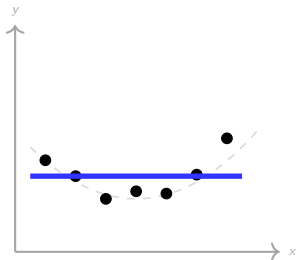
A model can score perfectly on training data yet **fail completely on new data**. Today we understand **why** this happens and **how to fix it**.

"Remember why we held data out in Day 2? Now you see what happens when you don't."

The Generalization Problem

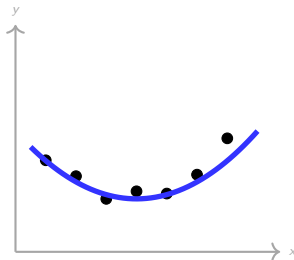
All three models see the **same training points** but fit them differently. Which one should we pick?

Model A



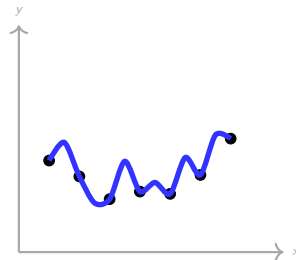
Train MSE ≈ 0.20

Model B



Train MSE ≈ 0.06

Model C

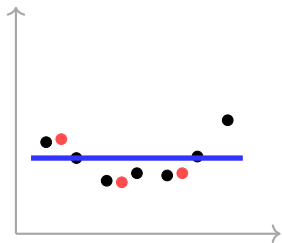


Train MSE ≈ 0.00

What Do We Really Want?

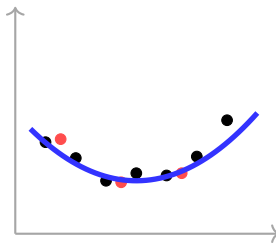
We care about how the model behaves on **new unseen data**. Imagine we collect fresh **red** samples:

Model A



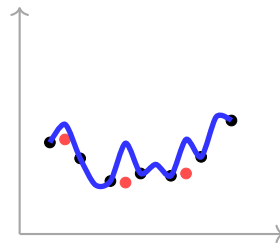
**Too Simple
(High Bias)**

Model B



**Captures Structure
(Balanced)**

Model C



**Too Complex
(High Variance)**

Definition

Generalization is the ability of a model to perform well on data it has **never seen before**.

High Training Accuracy $\neq$ Learning

- ▶ **Memorization (Capacity):** Modern models are powerful enough to memorize the entire training set, acting like a “lookup table” rather than a learner.
- ▶ **Shortcuts (Laziness):** If a simple noise pattern correlates with the target, the model will learn that instead of the true signal.

Domain	Intended Pattern	What Model Actually Learned
Vision	Animal features (ears, snout)	Snow background = Wolf
Tabular	Financial history	Application font = High Risk

Decomposition of Test Error

$$\text{Test Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$$

Bias

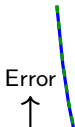
Error from wrong assumptions.
High bias $\rightarrow$ underfitting.

"How far is the average prediction from the truth?"

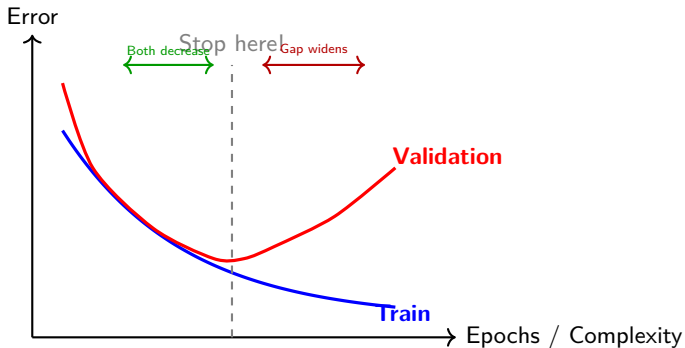
Variance

Error from sensitivity to training data.
High variance $\rightarrow$ overfitting.

"How much do predictions change across different training sets?"



How to Detect Overfitting



Key signal: Training error keeps decreasing, but validation error starts **increasing**. The growing gap = overfitting.

Imagine we train two linear models on the same data. Both achieve **Training MSE = 0.10**. Which one is better?

Model A (Small Weights)

$$y = 0.5x_1 + 0.3x_2$$

Val MSE: 0.12

Stable. Why?

→ Small changes in x cause
Small changes in y .

Model B (Huge Weights)

$$y = 500.5x_1 - 429.7x_2$$

Val MSE: 15.0 (!!!)

Unstable. Why?

→ Small changes in x cause
Massive changes in y .

Observation: Large weights are often a symptom of **Overfitting**. The model is trying too hard to wiggle through every training point.

The Fix: Constraining the Weights

We must force the model to keep weights small. We do this by adding a **Penalty** to the loss function.

$$\begin{array}{ccc} \text{Minimize Error} & + & \text{Minimize Weights} \\ \text{Fit the Data} & & \text{Keep Weights Small} \\ J_{final} = & \underbrace{J_{MSE}}_{\text{Fit the Data}} + & \underbrace{\lambda \cdot R(\mathbf{w})}_{\text{Keep Weights Small}} \end{array}$$

What is λ (Lambda)?

Regularization Strength (Hyperparameter):

- ▶ **High** λ : Huge penalty $\rightarrow$ Tiny weights (Model becomes simple) $\rightarrow$ Increases underfitting.
- ▶ **Low** λ : Small penalty $\rightarrow$ Weights can grow (Model fits data closely) $\rightarrow$ Increases overfitting.
- ▶ $\lambda = 0$: Standard Linear Regression (No penalty).

Let's set $\lambda = 1$ and Penalty $R(\mathbf{w}) = w^2$. Compare two models that have similar error, but different weights.

Model A (Overfit)

Weight $w = 100$

MSE Error = 0 (Perfect fit)

Total Loss:

$$J = 0 + 1 \cdot (100)^2$$

$$\mathbf{J} = 10,000$$

HUGE LOSS!

Model B (Balanced)

Weight $w = 2$

MSE Error = 5 (Some error)

Total Loss:

$$J = 5 + 1 \cdot (2)^2$$

$$\mathbf{J} = 9$$

WINNER!

We penalize the **Squared Magnitude** of the weights.

Objective Function

$$J(\mathbf{w}) = \text{MSE} + \lambda \sum w_j^2$$

Effect: “Weight Decay”

- ▶ Shrinks weights smoothly toward 0 (stability).
- ▶ Usually does not make weights exactly 0 (not sparse).
- ▶ Works well with correlated features.

Ridge Normal Equation

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$$

We penalize the **Absolute Value** of the weights.

Objective Function

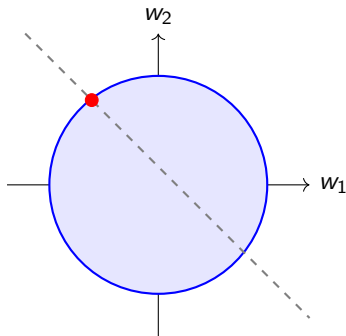
$$J(\mathbf{w}) = \text{MSE} + \lambda \sum |w_j|$$

Effect: “Sparsity”

- ▶ Encourages **sparsity**: many weights become exactly 0.
- ▶ Acts as **automatic feature selection**.
- ▶ Good when we believe **only a few features matter**.

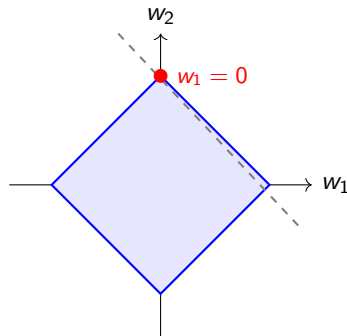
No closed-form solution (absolute value is not differentiable at 0). Must use **iterative optimization**.

L1 vs L2: Geometric Intuition



Loss Contour + L2 Constraint

Ridge: Smooth solution, rarely hits axis.
 $w_1 = -1.5, w_2 = 2.0$



Loss Contour + L1 Constraint

Lasso: Sharp corners promote sparsity.
 $w_1 = 0, w_2 = 2.5$

	Ridge (L2)	Lasso (L1)
Penalty	$\lambda \sum w_j^2$	$\lambda \sum w_j $
Constraint Shape	Circle	Diamond
Sparsity	No (weights ≈ 0)	Yes (weights = 0)
Feature Selection	No	Yes
Closed-Form	Yes	No
Best When	Many useful features	Few important features

Elastic Net combines both: $J = \text{MSE} + \lambda_1 \sum |w_j| + \lambda_2 \sum w_j^2$

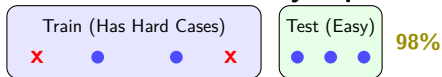
The Remaining Problem

Shuffling fixes the order bias, but with small datasets a single random split can still be **unlucky**.

The Law of Small Numbers:

- ▶ 20% of a small dataset is a tiny sample
- ▶ It may not match the population
- ▶ We get “Lucky” or “Unlucky” splits

Scenario A: “Lucky” Split



Scenario B: “Unlucky” Split



Same data, just a different random cut!

The Solution: K-Fold Cross-Validation

Use **ALL** data for testing (across multiple rounds):

1. Split shuffled data into K equal folds (e.g., $K = 5$)
2. Train on $K-1$ folds, test on the remaining 1
3. **Rotate** until every fold has been the test set exactly once
4. **Average** the K scores

Iter 5					Validation	Score: 0.86
Iter 4				Validation		Score: 0.85
Iter 3			Validation			Score: 0.84
Iter 2		Validation				Score: 0.83
Iter 1	Validation					Score: 0.82



Average Score: 0.84

- ▶ Same idea as K-Fold, but each fold preserves the **class distribution**
- ▶ Combines the benefits of K-Fold (robust evaluation) with stratification (balanced classes)

Code

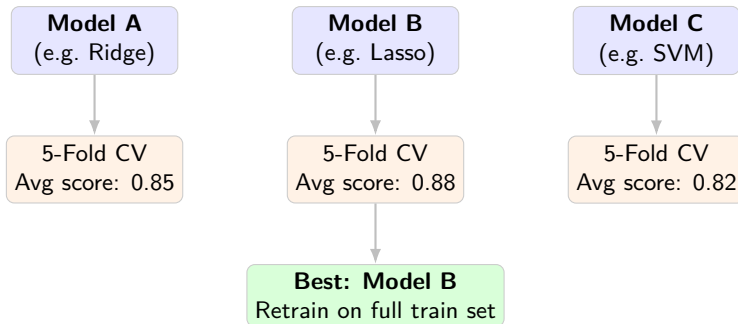
```
from sklearn.model_selection import StratifiedKFold
skf = StratifiedKFold(n_splits=5, shuffle=True)
for train_idx, test_idx in skf.split(X, y):
    X_train, X_test = X[train_idx], X[test_idx]
    # train model, compute score ...
```

- ▶ **Default choice** for classification — use this unless you have a reason not to

- ▶ Sometimes samples are **not independent** — they belong to groups
 - e.g. multiple medical images from the **same patient**
 - e.g. several transactions from the **same user**
- ▶ If the same patient appears in both train and test, the model can “cheat”
- ▶ **Group K-Fold** ensures all samples from one group are in the **same fold**

Code

```
from sklearn.model_selection import GroupKFold
gkf = GroupKFold(n_splits=5)
for train_idx, test_idx in gkf.split(X, y,
groups=patient_ids):
    # no patient appears in both train and test
```



Use CV scores to **pick the best model / hyperparameters**, then retrain the winner on the **entire training set** before evaluating on the held-out test set.

Parameters (θ)

Learned from data during training.

- ▶ Weights $\mathbf{w}$ in linear regression
- ▶ Support vectors in SVM
- ▶ Split thresholds in decision trees

Set by the **optimizer**.

Hyperparameters

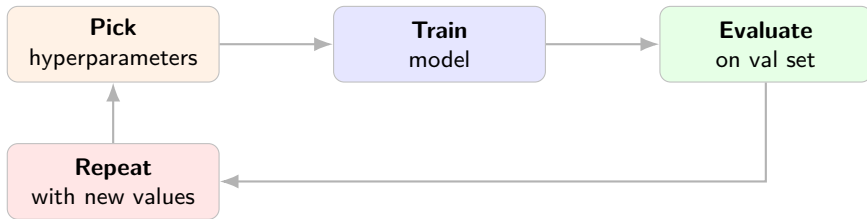
Set by us before training.

- ▶ Learning rate α
- ▶ Regularization λ
- ▶ Number of folds k
- ▶ Tree depth, # estimators

Set by **tuning / search**.

Key insight: Hyperparameters control model complexity. Choosing them well is critical to avoid overfitting / underfitting.

Goal: Find the hyperparameter combination that gives the best **validation** performance.



How do we choose which combinations to try?

1. **Grid Search** — try every combination
2. **Random Search** — sample randomly

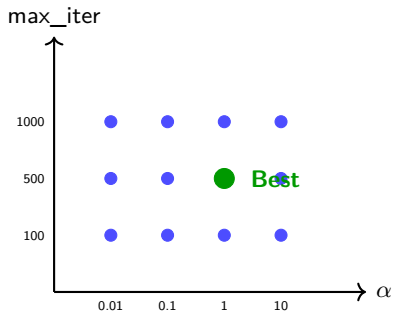
Idea: Define a grid of values for each hyperparameter and try **every combination**.

```
from sklearn.model_selection
import GridSearchCV

param_grid = {
    'alpha': [0.01, 0.1, 1, 10],
    'max_iter': [100, 500, 1000]
}

grid = GridSearchCV(
    model, param_grid, cv=5)
grid.fit(X_train, y_train)
```

$4 \times 3 = 12$ combinations $\times$ 5 folds = 60 fits



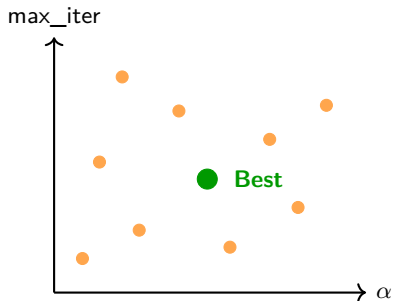
Exhaustive but **expensive**.
Cost grows exponentially with #
hyperparameters.

Idea: Sample hyperparameters **randomly** from distributions. Often finds good values faster.

```
from sklearn.model_selection
import RandomizedSearchCV

param_dist = {
    'alpha': uniform(0.001, 10),
    'max_iter': [100, 500, 1000]
}

search = RandomizedSearchCV(
    model, param_dist,
    n_iter=20, cv=5)
search.fit(X_train, y_train)
```



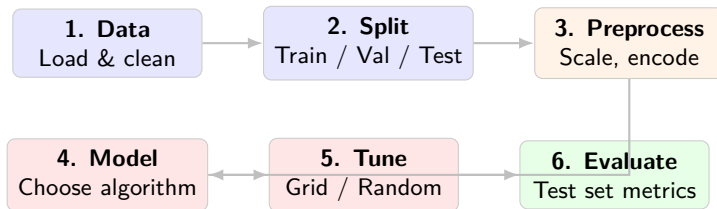
Explores **more of the space**.
Better when some hyperparameters
matter more than others.

	Grid Search	Random Search
Strategy	Exhaustive	Sampling
Coverage	Uniform grid points	Spread across space
Cost	$O(\prod n_i)$	$O(n_iter)$
Scales to many HPs	Poorly	Well
Guarantees optimum	In the grid, yes	No

Practical advice:

- ▶ Start with **Random Search** to find a good region.
- ▶ Then refine with **Grid Search** in that region.

Putting It All Together: The ML Pipeline



Golden rule: The **test set** is touched **only once** — at the very end.
All tuning and model selection happens on **training + validation** data.

Concept	Key Takeaway
Overfitting	Model memorizes noise; high variance
Underfitting	Model too simple; high bias
Bias-Variance	Minimize total error at the sweet spot
Ridge (L2)	Shrinks weights smoothly; no sparsity
Lasso (L1)	Drives weights to zero; feature selection
Hyperparameters	Set before training; control complexity
Grid Search	Exhaustive; reliable but expensive
Random Search	Efficient; scales to many hyperparameters

Next: Classical ML Models — KNN, SVM, Decision Trees, Ensembles

Credits

KAUST Academy

King Abdullah University of Science and Technology

KAUST–Mawhiba IOAI 2026 Training Program